Name: Remy Nteziryayo
ID: 90102028
Cohort: B

===========================================

```
MariaDB [(none)]> create database project_db;
Query OK, 1 row affected (0.030 sec)

MariaDB [(none)]> use project_db;
Database changed
MariaDB [project_db]>
```

--Exercise 7.24 - Creating the Projects Schema

---

```
MariaDB [project_db]> CREATE TABLE Employee (
    ->     empNo VARCHAR(10) PRIMARY KEY,
    ->     fName VARCHAR(50),
    ->     lName VARCHAR(50),
    ->     address VARCHAR(255),
    ->     DOB DATE,
    ->     sex CHAR(1) CHECK (sex IN ('M', 'F')),
    ->     position VARCHAR(50) CHECK (position IN ('Manager', 'Team Leader',
'Analyst', 'Software Developer')),
    ->     deptNo VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.078 sec)


MariaDB [project_db]> CREATE TABLE Department (
    ->     deptNo VARCHAR(10) PRIMARY KEY,
    ->     deptName VARCHAR(100),
    ->     mgrEmpNo VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.015 sec)


MariaDB [project_db]> CREATE TABLE Project (
    ->     projNo VARCHAR(10) PRIMARY KEY,
    ->     projName VARCHAR(100),
    ->     deptNo VARCHAR(10),
    ->     FOREIGN KEY (deptNo) REFERENCES Department(deptNo)
    -> );
Query OK, 0 rows affected (0.027 sec)

MariaDB [project_db]> CREATE TABLE WorksOn (
    ->     empNo VARCHAR(10),
    ->     projNo VARCHAR(10),
```

```
    ->     dateWorked DATE,
    ->     hoursWorked INT CHECK (hoursWorked >= 0 AND hoursWorked <= 40),
    ->     PRIMARY KEY (empNo, projNo, dateWorked),
    ->     FOREIGN KEY (empNo) REFERENCES Employee(empNo),
    ->     FOREIGN KEY (projNo) REFERENCES Project(projNo)
    -> );
Query OK, 0 rows affected (0.038 sec)


--Because SQL executes commands one after the other
--neither table can be created first if they both strictly require the other to
already exist.
-- that is why I am altering the table

MariaDB [project_db]> ALTER TABLE Employee
    -> ADD FOREIGN KEY (deptNo) REFERENCES Department(deptNo);
Query OK, 0 rows affected (0.067 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [project_db]> ALTER TABLE Department
    -> ADD FOREIGN KEY (mgrEmpNo) REFERENCES Employee(empNo);
Query OK, 0 rows affected (0.052 sec)
Records: 0  Duplicates: 0  Warnings: 0

-- Disable FK checks temporarily to insert circular data
MariaDB [project_db]> SET FOREIGN_KEY_CHECKS = 0;
Query OK, 0 rows affected (0.028 sec)

MariaDB [project_db]> INSERT INTO Employee (empNo, fName, lName, address, DOB, sex,
position, deptNo) VALUES
    -> ('E1', 'John', 'Doe', '123 Elm St', '1980-01-01', 'M', 'Team Leader', 'D1'),
    -> ('E2', 'Jane', 'Smith', '456 Oak St', '1975-05-15', 'F', 'Manager', 'D1'), --
Female Manager
    -> ('E3', 'Bob', 'Jones', '789 Pine St', '1990-08-20', 'M', 'Software
Developer', 'D2'),
    -> ('E4', 'Alice', 'Brown', '321 Maple St', '1985-11-30', 'F', 'Manager', 'D2');
-- Female Manager
Query OK, 4 rows affected (0.039 sec)
Records: 4  Duplicates: 0  Warnings: 0



MariaDB [project_db]> INSERT INTO Department (deptNo, deptName, mgrEmpNo) VALUES
    -> ('D1', 'IT', 'E2'),
    -> ('D2', 'HR', 'E4');
Query OK, 2 rows affected (0.029 sec)
Records: 2  Duplicates: 0  Warnings: 0


MariaDB [project_db]> INSERT INTO Project (projNo, projName, deptNo) VALUES
```

```
    -> ('SCCS', 'System Control', 'D1'),
    -> ('P2', 'Payroll Upgrade', 'D2');
Query OK, 2 rows affected (0.028 sec)
Records: 2  Duplicates: 0  Warnings: 0


MariaDB [project_db]> INSERT INTO WorksOn (empNo, projNo, dateWorked, hoursWorked)
VALUES
    -> ('E1', 'SCCS', '2025-10-01', 10),
    -> ('E1', 'P2', '2025-10-02', 20),
    -> ('E2', 'SCCS', '2025-10-01', 15),
    -> ('E3', 'P2', '2025-10-03', 40);
Query OK, 4 rows affected (0.029 sec)
Records: 4  Duplicates: 0  Warnings: 0


MariaDB [project_db]> SET FOREIGN_KEY_CHECKS = 1;
Query OK, 0 rows affected (0.001 sec)
```

--Exercise 7.25 - Creating the Female Managers View

---

```
MariaDB [project_db]> CREATE VIEW FemaleManagedProjects AS
    -> SELECT p.projNo, p.projName, p.deptNo
    -> FROM Project p
    -> JOIN Department d ON p.deptNo = d.deptNo
    -> JOIN Employee e ON d.mgrEmpNo = e.empNo
    -> WHERE e.sex = 'F'
    -> ORDER BY p.projNo;
Query OK, 0 rows affected (0.043 sec)
-- test view
MariaDB [project_db]> SELECT * FROM FemaleManagedProjects;
+--------+-----------------+--------+
| projNo | projName        | deptNo |
+--------+-----------------+--------+
| P2     | Payroll Upgrade | D2     |
| SCCS   | System Control  | D1     |
+--------+-----------------+--------+
2 rows in set (0.042 sec)
```

--Exercise 7.26 - Creating the Employee Project Details View

---

```
MariaDB [project_db]> CREATE VIEW EmployeeProjectHours AS
    -> SELECT e.empNo, e.fName, e.lName, p.projName, w.hoursWorked
    -> FROM Employee e
    -> JOIN WorksOn w ON e.empNo = w.empNo
    -> JOIN Project p ON w.projNo = p.projNo;
```

```
Query OK, 0 rows affected (0.008 sec)

MariaDB [project_db]> SELECT * FROM EmployeeProjectHours;
+-------+-------+-------+----------------+-------------+
| empNo | fName | lName | projName       | hoursWorked |
+-------+-------+-------+----------------+-------------+
| E1    | John  | Doe   | Payroll Upgrade |          20 |
| E3    | Bob   | Jones | Payroll Upgrade |          40 |
| E1    | John  | Doe   | System Control |          10 |
| E2    | Jane  | Smith | System Control |          15 |
+-------+-------+-------+----------------+-------------+
4 rows in set (0.003 sec)


--Exercise 7.27 - Analyzing the EmpProject View
___________________________________________________

MariaDB [project_db]> CREATE VIEW EmpProject(empNo, projNo, totalHours)
    -> AS SELECT w.empNo, w.projNo, SUM(hoursWorked)
    -> FROM Employee e, Project p, WorksOn w
    -> WHERE e.empNo = w.empNo AND p.projNo = w.projNo
    -> GROUP BY w.empNo, w.projNo;
Query OK, 0 rows affected (0.034 sec)



MariaDB [project_db]> SELECT * FROM EmpProject;
+-------+--------+------------+
| empNo | projNo | totalHours |
+-------+--------+------------+
| E1    | P2     |         20 |
| E1    | SCCS   |         10 |
| E2    | SCCS   |         15 |
| E3    | P2     |         40 |
+-------+--------+------------+
4 rows in set (0.031 sec)

Analyis:
________
This query is valid. It returns a summary of all employees,
showing the total hours they have aggregated per project.

MariaDB [project_db]> SELECT projNo FROM EmpProject WHERE projNo = 'SCCS';
+--------+
| projNo |
+--------+
| SCCS   |
| SCCS   |
+--------+
2 rows in set (0.029 sec)
```

Analysis:

________
This query is valid. It filters the view to only show rows where the project number is 'SCCS',
returning that project number for every employee who worked on it.

```
MariaDB [project_db]> SELECT COUNT(projNo) FROM EmpProject WHERE empNo = 'E1';
+---------------+
| COUNT(projNo) |
+---------------+
|             2 |
+---------------+
1 row in set (0.002 sec)
```

Analyis:

________
This query is valid. It counts the number of distinct projects that
employee 'E1' has worked on by counting the rows returned for that employee.

```
MariaDB [project_db]> SELECT empNo, totalHours FROM EmpProject GROUP BY empNo;
+-------+------------+
| empNo | totalHours |
+-------+------------+
| E1    |         20 |
| E2    |         15 |
| E3    |         40 |
+-------+------------+
3 rows in set (0.028 sec)
```

Analyis:

________
This query is has some logic error.
It groups by empNo but selects totalHours without an aggregate function (like SUM()).

```
-- this is to correct the above Query because it is not returning the full hours
-- we need the SUM function.

MariaDB [project_db]> SELECT empNo, SUM(totalHours) AS overallTotalHours
    -> FROM EmpProject
    -> GROUP BY empNo;
+-------+-------------------+
| empNo | overallTotalHours |
+-------+-------------------+
| E1    |                30 |
| E2    |                15 |
| E3    |                40 |
```

```
+-------+------------------+
3 rows in set (0.002 sec)
```

Exercise 7.28 - Maintaining a Materialized View

since A materialized view  stores the results of a query.
 It improves performance because the data is already saved,
  but it must be updated when the base table changes.

The materialized view stores all different partNo values
 where partCost is more than £1000. When the Part table
 changes, the view must also be updated.
 Inserts and updates that make the cost above £1000
 can be updated directly. For deletes or when the cost
 becomes £1000 or less, the Part table must be checked
 because another row with the same partNo may still
 have a cost above £1000.